

10. Calculer : + - * / et %

Diviser des entiers laisse un reste — et ce reste est souvent ce qu'on cherche.

1. Ce qu'il faut comprendre

Les cinq opérations de base : `+` `-` `*` `/` et `%`. Les trois premières sont celles de l'école. Les deux dernières demandent de l'attention, parce qu'une variable ne contient que des **entiers**.

`/` est la **division entière** : `200 / 60` vaut **3**, pas 3,33. Ce qui dépasse est jeté. La console n'a pas de nombres à virgule ; un jeu Game Boy compte en entiers, et en petites unités (des pixels, des images) plutôt qu'en fractions.

`%` (« modulo ») donne le **reste** de cette division : `200 % 60` vaut **20**. Deux cents images, c'est 3 secondes **et** 20 images. Le modulo sert partout dans un jeu : « une fois sur quatre » (`n % 4 == 0`), « tourner en rond » dans une liste, séparer des chiffres.

Les **priorités** sont celles des mathématiques : `*` `/` `%` passent avant `+` `-`. `2 + 3 * 4` vaut 14. Les parenthèses décident quand on veut autre chose : `(2 + 3) * 4` vaut 20.

À toi : calcule combien de minutes et de secondes font 150 secondes — avec `/` et `%` sur 60.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)
- `#include <diviser>` → [0.8.5. #include <diviser> — a / b](#)
- `#include <reste>` → [0.8.4. #include <reste> — a % b](#)
- `#include <multiplier>` → [9.1. #include <multiplier> — a * b, quand les deux se calculent](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **994 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage — la mise en route de la console
main()	187 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte, EcrireNombre — attend de pouvoir écrire une case
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
EcrireNombre	33 o	← nombre() — écrit un nombre en chiffres
Diviser	17 o	← main() — divise — le processeur ne sait pas le faire
Reste	10 o	← main() — le reste d'une division
Multiplier	14 o	← main() — multiplie — le processeur ne sait pas le faire
CopierTuiles	112 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	432 o	← CopierTuiles — 27 lettres sur 43
"IMAGES:"	7 o	← texte() — un texte gravé

```

"SECONDES:"      9 o ← texte() – un texte gravé
"ET IMAGES:"     10 o ← texte() – un texte gravé
"TOTAL:"         6 o ← texte() – un texte gravé
"SANS PAR.:"     10 o ← texte() – un texte gravé
"AVEC PAR.:"     10 o ← texte() – un texte gravé
Lettres gravées : ACDEGILMNOPRSTV0123456789.:
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, DecalerGauche, DecalerDroite,
RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes
Fonctions du langage appelées :
    texte() × 6    66 o sur place, dans main() + EcrireTexte (10 o)
    nombre() × 6   60 o sur place, dans main() + EcrireNombre (33 o)
    image() × 1    3 o sur place, dans main() + AttendreImage (28 o)

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// CE PROGRAMME : fait des calculs avec + - * / et %, puis affiche les résultats.
//
// Rappel : « // » commence un commentaire, que la console ignore.
// uint8_t = une case de mémoire qui garde un nombre entier de 0 à 255.

#include <texte>      // écrit un texte à l'écran
#include <nombre>     // écrit un nombre en chiffres
#include <diviser>    // a / b, sauf par 1, 2, 4, 8, 16... écrits en clair
#include <reste>      // a % b, sauf par 1, 2, 4, 8, 16... écrits en clair
#include <multiplier> // a * b, quand les deux se calculent

```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```

int main() {
    uint8_t duree = 200; // Le jeu commence ici.
                        // 200 images. La console en montre 60 par seconde.

    uint8_t secondes = duree / 60; // « / » = la DIVISION ENTIÈRE : 200 / 60 = 3.
                                // (3 fois 60 = 180 ; ce qui dépasse est jeté,
                                // il n'y a pas de nombres à virgule.)

    uint8_t reste = duree % 60; // « % » = le RESTE de cette division (on dit « modulo ») :
                                // 200 - 180 = 20. Donc 200 images = 3 secondes et 20 images.

    uint8_t pieces = 7; // 7 pièces...
    uint8_t prix = 12; // ... à 12 chacune...
    uint8_t total = pieces * prix; // ... « * » = multiplier : 7 fois 12 = 84.

    uint8_t calcul = 2 + 3 * 4; // La multiplication passe AVANT l'addition :
                                // 3 * 4 = 12, puis 2 + 12 = 14.
    uint8_t autre = (2 + 3) * 4; // Les parenthèses passent avant tout :
                                // 2 + 3 = 5, puis 5 * 4 = 20.

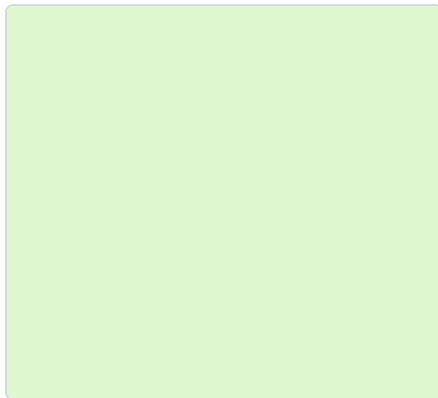
    // On affiche tout. texte(colonne, ligne, "MOT") écrit un mot ;
    // nombre(colonne, ligne, valeur) écrit un nombre sur 3 chiffres.
    // On peut mettre deux instructions sur la même ligne : chacune finit par « ; ».
    texte(1, 2, "IMAGES:"); nombre(14, 2, duree); // 200
    texte(1, 4, "SECONDES:"); nombre(14, 4, secondes); // 003
    texte(1, 5, "ET IMAGES:"); nombre(14, 5, reste); // 020
    texte(1, 8, "TOTAL:"); nombre(14, 8, total); // 084
    texte(1, 11, "SANS PAR.:"); nombre(14, 11, calcul); // 014
    texte(1, 12, "AVEC PAR.:"); nombre(14, 12, autre); // 020

    while (true) { // Répète pour toujours...
        image(); // ... attendre l'image suivante. Le jeu reste allumé.
    }
}

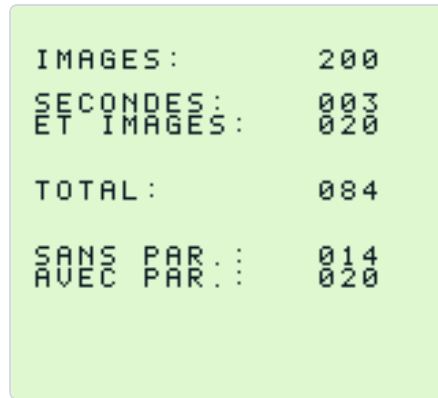
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

200 images = 3 secondes et 20 images ; $7 \times 12 = 84$; 14 sans parenthèses, 20 avec.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ 200 / 60 donne 3 (division entière)

✓ 200 % 60 donne le reste, 20

✓ 7 * 12 donne 84

✓ 2 + 3 * 4 donne 14

✓ (2 + 3) * 4 donne 20

Ensuite : **11. Un octet s'arrête à 255** — chapitre 2 : Les variables : la mémoire de la console. · gameboy3 — écrire une cartouche Game Boy en C++